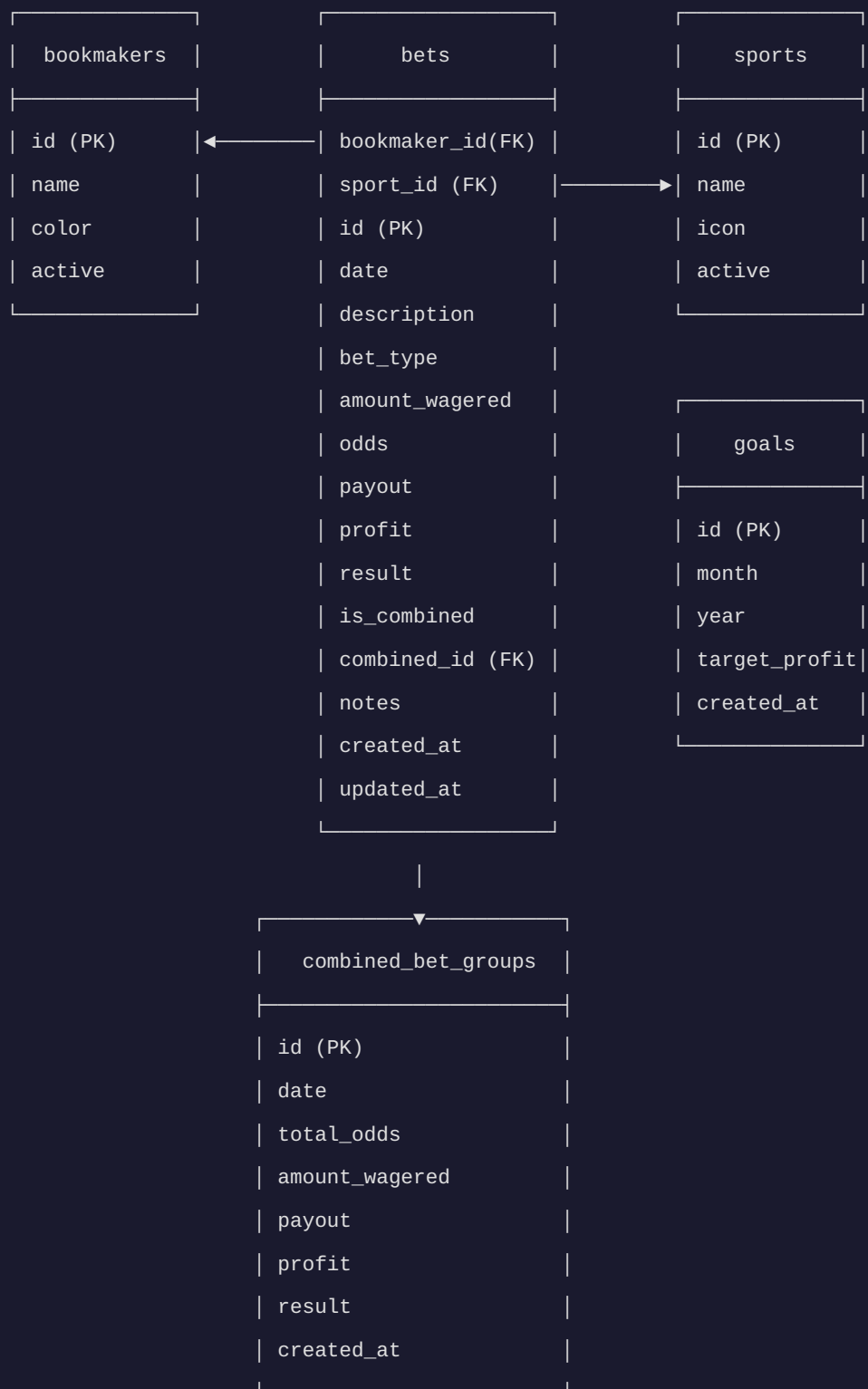


Schema do Banco de Dados — BetManager

Diagrama de Entidade-Relacionamento





Tabelas

sports — Esportes

```
CREATE TABLE sports (  
    id          SERIAL PRIMARY KEY,  
    name        VARCHAR(100) NOT NULL UNIQUE,  
    icon        VARCHAR(50),           -- emoji ou nome de ícone  
    active      BOOLEAN DEFAULT TRUE,  
    created_at  TIMESTAMP DEFAULT NOW()  
);  
  
-- Dados iniciais  
INSERT INTO sports (name, icon) VALUES  
    ('Futebol', '⚽'),  
    ('Basquete', '🏀'),  
    ('Tênis', '🎾'),  
    ('Fórmula 1', '🏎️'),  
    ('MMA/UFC', '🥊'),  
    ('Voleibol', '🏐'),  
    ('Beisebol', '⚾'),  
    ('Hóquei', '🏒'),  
    ('Outro', '🎮');
```

```
CREATE TABLE bookmakers (  
    id          SERIAL PRIMARY KEY,  
    name        VARCHAR(100) NOT NULL UNIQUE,  
    color       VARCHAR(7) DEFAULT '#6B7280',    -- hex para exibição no frontend  
    active      BOOLEAN DEFAULT TRUE,  
    created_at  TIMESTAMP DEFAULT NOW()  
);  
  
-- Dados iniciais  
INSERT INTO bookmakers (name, color) VALUES  
    ('Bet365',    '#007B5E'),  
    ('Superbet', '#E63946'),  
    ('Betano',    '#FF6B35'),  
    ('Sportingbet', '#1A1A2E'),  
    ('KTO',       '#FFD700'),  
    ('Betfair',   '#FFD700'),  
    ('Pinnacle',  '#003087'),  
    ('Outros',    '#6B7280');
```

`combined_bet_groups` — Grupos de Apostas Combinadas

```
CREATE TABLE combined_bet_groups (  
    id                SERIAL PRIMARY KEY,  
    date              DATE NOT NULL,  
    total_odds        NUMERIC(10, 4),          -- produto das odds individuais  
    amount_wagered    NUMERIC(10, 2) NOT NULL,  
    payout            NUMERIC(10, 2) DEFAULT 0,  
    profit            NUMERIC(10, 2)          -- calculado: payout - amount_wagered  
                    GENERATED ALWAYS AS (payout - amount_wagered) STORED,  
    result            VARCHAR(20) NOT NULL     -- 'won' | 'lost' | 'void' | 'pending'  
                    CHECK (result IN ('won', 'lost', 'void', 'pending')),  
    notes             TEXT,  
    created_at        TIMESTAMP DEFAULT NOW(),  
    updated_at        TIMESTAMP DEFAULT NOW()  
);
```

bets — Apostas (tabela principal)

```

CREATE TABLE bets (
    id SERIAL PRIMARY KEY,

    -- Relacionamentos
    sport_id INTEGER REFERENCES sports(id) ON DELETE SET NULL,
    bookmaker_id INTEGER NOT NULL REFERENCES bookmakers(id) ON DELETE RESTRICT,
    combined_id INTEGER REFERENCES combined_bet_groups(id) ON DELETE SET NULL,

    -- Dados da aposta
    date DATE NOT NULL,
    description VARCHAR(500) NOT NULL, -- ex: "Nneka 017.5P", "Brasil 02.5 HT"
    bet_type VARCHAR(20) NOT NULL -- 'simple' | 'combined'
        CHECK (bet_type IN ('simple', 'combined')),
    is_combined BOOLEAN DEFAULT FALSE, -- true se faz parte de uma combinada

    -- Valores financeiros (NUMERIC para precisão monetária – nunca FLOAT)
    amount_wagered NUMERIC(10, 2) NOT NULL
        CHECK (amount_wagered > 0),
    odds NUMERIC(10, 4)
        CHECK (odds IS NULL OR odds >= 1.0),
    payout NUMERIC(10, 2) DEFAULT 0
        CHECK (payout >= 0),
    profit NUMERIC(10, 2) -- calculado: payout - amount_wagered
        GENERATED ALWAYS AS (payout - amount_wagered) STORED,

    -- Resultado
    result VARCHAR(20) NOT NULL DEFAULT 'pending'
        CHECK (result IN ('won', 'lost', 'void', 'pending')),

    -- Metadados
    notes TEXT,
    source VARCHAR(20) DEFAULT 'manual' -- 'manual' | 'ai_extract'
        CHECK (source IN ('manual', 'ai_extract')),
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Índices para performance das queries analíticas

```

```
CREATE INDEX idx_bets_date          ON bets(date DESC);
CREATE INDEX idx_bets_result        ON bets(result);
CREATE INDEX idx_bets_sport_id      ON bets(sport_id);
CREATE INDEX idx_bets_bookmaker_id  ON bets(bookmaker_id);
CREATE INDEX idx_bets_bet_type      ON bets(bet_type);
CREATE INDEX idx_bets_date_result   ON bets(date DESC, result); -- query mais comum do dashboard
```

goals — Metas

```
CREATE TABLE goals (
  id          SERIAL PRIMARY KEY,
  month       SMALLINT NOT NULL CHECK (month BETWEEN 1 AND 12),
  year        SMALLINT NOT NULL CHECK (year >= 2024),
  target_profit NUMERIC(10, 2) NOT NULL CHECK (target_profit > 0),
  notes       TEXT,
  created_at  TIMESTAMP DEFAULT NOW(),
  updated_at  TIMESTAMP DEFAULT NOW(),

  CONSTRAINT unique_month_year UNIQUE (month, year)
);
```

ai_extraction_logs — Log de Extrações por IA

```
CREATE TABLE ai_extraction_logs (
  id          SERIAL PRIMARY KEY,
  model_used  VARCHAR(50) NOT NULL,      -- ex: 'claude-haiku-3'
  bets_detected INTEGER DEFAULT 0,      -- quantas apostas a IA encontrou
  bets_confirmed INTEGER DEFAULT 0,    -- quantas o usuário confirmou
  input_tokens INTEGER,
  output_tokens INTEGER,
  success     BOOLEAN DEFAULT TRUE,
  error_message TEXT,
  created_at  TIMESTAMP DEFAULT NOW()
);
```

Views Analíticas

`v_dashboard_monthly` — Dados mensais do dashboard

```
CREATE VIEW v_dashboard_monthly AS
SELECT
    DATE_TRUNC('month', date)          AS month,
    COUNT(*)                          AS total_bets,
    COUNT(*) FILTER (WHERE result = 'won') AS won,
    COUNT(*) FILTER (WHERE result = 'lost') AS lost,
    COUNT(*) FILTER (WHERE result = 'void') AS void,
    SUM(amount_wagered)                AS total_wagered,
    SUM(payout)                        AS total_payout,
    SUM(profit)                        AS total_profit,
    ROUND(
        COUNT(*) FILTER (WHERE result = 'won')::NUMERIC /
        NULLIF(COUNT(*) FILTER (WHERE result IN ('won', 'lost')), 0) * 100, 2
    )                                  AS hit_rate_pct,
    ROUND(AVG(odds) FILTER (WHERE odds IS NOT NULL), 4) AS avg_odds
FROM bets
WHERE result != 'pending'
GROUP BY DATE_TRUNC('month', date)
ORDER BY month DESC;
```

v_stats_by_sport — Estatísticas por esporte

```
CREATE VIEW v_stats_by_sport AS
SELECT
    s.name                AS sport,
    s.icon,
    COUNT(b.id)           AS total_bets,
    COUNT(*) FILTER (WHERE b.result = 'won') AS won,
    COUNT(*) FILTER (WHERE b.result = 'lost') AS lost,
    SUM(b.amount_wagered) AS total_wagered,
    SUM(b.profit)          AS total_profit,
    ROUND(AVG(b.profit), 2) AS avg_profit_per_bet,
    ROUND(
        COUNT(*) FILTER (WHERE b.result = 'won')::NUMERIC /
        NULLIF(COUNT(*) FILTER (WHERE b.result IN ('won', 'lost')), 0) * 100, 2
    ) AS hit_rate_pct
FROM bets b
JOIN sports s ON b.sport_id = s.id
WHERE b.result != 'pending'
GROUP BY s.id, s.name, s.icon
ORDER BY total_profit DESC;
```

v_stats_by_bookmaker — Estatísticas por casa de apostas

```
CREATE VIEW v_stats_by_bookmaker AS
SELECT
    bk.name                AS bookmaker,
    bk.color,
    COUNT(b.id)            AS total_bets,
    COUNT(*) FILTER (WHERE b.result = 'won') AS won,
    COUNT(*) FILTER (WHERE b.result = 'lost') AS lost,
    SUM(b.amount_wagered)  AS total_wagered,
    SUM(b.profit)           AS total_profit,
    ROUND(AVG(b.profit), 2) AS avg_profit_per_bet,
    ROUND(
        COUNT(*) FILTER (WHERE b.result = 'won')::NUMERIC /
        NULLIF(COUNT(*) FILTER (WHERE b.result IN ('won', 'lost')), 0) * 100, 2
    ) AS hit_rate_pct
FROM bets b
JOIN bookmakers bk ON b.bookmaker_id = bk.id
WHERE b.result != 'pending'
GROUP BY bk.id, bk.name, bk.color
ORDER BY total_profit DESC;
```

v_stats_by_bet_type — Simples vs Combinadas

```
CREATE VIEW v_stats_by_bet_type AS
SELECT
    bet_type,
    COUNT(*) AS total_bets,
    COUNT(*) FILTER (WHERE result = 'won') AS won,
    COUNT(*) FILTER (WHERE result = 'lost') AS lost,
    SUM(amount_wagered) AS total_wagered,
    SUM(profit) AS total_profit,
    ROUND(AVG(profit), 2) AS avg_profit_per_bet,
    ROUND(
        COUNT(*) FILTER (WHERE result = 'won')::NUMERIC /
        NULLIF(COUNT(*) FILTER (WHERE result IN ('won', 'lost')), 0) * 100, 2
    ) AS hit_rate_pct
FROM bets
WHERE result != 'pending'
GROUP BY bet_type;
```

Regras de Negócio no Banco

Trigger: atualizar `updated_at` automaticamente

```
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_bets_updated_at
    BEFORE UPDATE ON bets
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();

CREATE TRIGGER trg_combined_updated_at
    BEFORE UPDATE ON combined_bet_groups
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();

CREATE TRIGGER trg_goals_updated_at
    BEFORE UPDATE ON goals
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

Trigger: sincronizar resultado de aposta combinada

Quando todas as seleções de uma combinada tiverem resultado, o grupo é atualizado automaticamente.

```

CREATE OR REPLACE FUNCTION sync_combined_result()
RETURNS TRIGGER AS $$
DECLARE
    v_combined_id INTEGER;
    v_all_resolved BOOLEAN;
    v_any_lost BOOLEAN;
    v_any_void BOOLEAN;
BEGIN
    v_combined_id := NEW.combined_id;
    IF v_combined_id IS NULL THEN RETURN NEW; END IF;

    SELECT
        NOT EXISTS (SELECT 1 FROM bets WHERE combined_id = v_combined_id AND result = 'pending'),
        EXISTS      (SELECT 1 FROM bets WHERE combined_id = v_combined_id AND result = 'lost'),
        EXISTS      (SELECT 1 FROM bets WHERE combined_id = v_combined_id AND result = 'void')
    INTO v_all_resolved, v_any_lost, v_any_void;

    IF v_all_resolved THEN
        UPDATE combined_bet_groups
        SET result = CASE
            WHEN v_any_lost THEN 'lost'
            WHEN v_any_void THEN 'void'
            ELSE 'won'
        END
        WHERE id = v_combined_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sync_combined_result
AFTER UPDATE OF result ON bets
FOR EACH ROW EXECUTE FUNCTION sync_combined_result();

```

Migrations (Alembic)

```
# Criar nova migration
alembic revision --autogenerate -m "descricao_da_mudanca"

# Aplicar migrations pendentes
alembic upgrade head

# Reverter última migration
alembic downgrade -1

# Ver histórico
alembic history
```

Convenções

- Todos os valores monetários usam `NUMERIC(10, 2)` — nunca `FLOAT` (evita erros de ponto flutuante)
- Odds usam `NUMERIC(10, 4)` para suportar odds fracionadas com precisão
- Datas sem hora usam `DATE` ; timestamps completos usam `TIMESTAMP` sem timezone (UTC implícito)
- Colunas `created_at` e `updated_at` em todas as tabelas mutáveis
- Foreign keys com `ON DELETE RESTRICT` para bookmakers (não pode deletar casa com apostas associadas)
- Foreign keys com `ON DELETE SET NULL` para sports (sport deletado não perde a aposta)